

AWS DEVOPS INTERVIEW GUIDE

Scenario 1: Deploy a Containerized Application on AWS Using ECS

Scenario

Your development team has containerized a Java Spring Boot application using Docker. They want to deploy it on AWS without managing Kubernetes. The application should be highly available and scalable.

How would you design and deploy this solution using Amazon ECS?

Solution

Architecture



Implementation Steps

1. Build the Docker image.
2. Push the image to Amazon ECR.
3. Create an ECS Cluster (Fargate or EC2).
4. Create a Task Definition:
 - CPU and Memory
 - Docker Image
 - Port Mapping
 - Environment Variables
 - IAM Task Role
5. Create an ECS Service.
6. Attach the service to an Application Load Balancer.
7. Configure Auto Scaling.

Why ECS?

- Fully managed container orchestration
- No Kubernetes management
- Easy integration with AWS services
- Supports rolling deployments
- High availability across multiple Availability Zones

Interview Points

- ECS Cluster
 - Task Definition
 - Task
 - Service
 - ALB Integration
 - Auto Scaling
-

Scenario 2: ECS Tasks Are Continuously Stopping

Scenario

After deployment, ECS tasks start successfully but stop after a few seconds. Users cannot access the application.

How would you troubleshoot?

Troubleshooting Approach

Step 1: Check ECS Task Status

```
aws ecs describe-tasks
```

Look for:

- Stopped Reason
 - Exit Code
-

Step 2: Check CloudWatch Logs

Common errors:

- Application crash
 - Missing configuration
 - Database connection failure
 - Port binding issues
-

Step 3: Verify Task Definition

Check:

- Image URI
 - CPU/Memory
 - Environment Variables
 - Port Mapping
-

Step 4: Verify IAM Role

Ensure the task has permission to:

- Pull images from ECR
 - Read Secrets Manager
 - Access S3
 - Write CloudWatch Logs
-

Step 5: Check Health Checks

If ALB health checks fail repeatedly:

ALB

↓

Task marked unhealthy

↓

Task stopped

↓

New Task launched

Verify:

- Health check path
- Port
- Startup time
- Application status

Common Causes

- Wrong image
 - Application crash
 - Memory exceeded (OOM)
 - CPU starvation
 - Missing secrets
 - Incorrect health check
 - Database unavailable
-

Scenario 3: Application Must Scale Automatically During Peak Hours

Scenario

Your application receives 10x more traffic during business hours. Manual scaling is not acceptable.

How would you configure ECS Auto Scaling?

Solution

Use **Application Auto Scaling**.

Architecture

```
Users
|
Application Load Balancer
|
ECS Service
|
Application Auto Scaling
```

Configure

Minimum Tasks:

2

Desired:

4

Maximum:

10

Scaling Policy

CPU > 70%

↓

Increase Tasks

Scale In

CPU < 30%

↓

Reduce Tasks

Metrics:

- CPU Utilization
- Memory Utilization
- ALB Request Count per Target
- Custom CloudWatch Metrics

Benefits

- High availability
- Cost optimization
- Automatic scaling
- No manual intervention

Scenario 4: Perform Zero-Downtime Deployment in ECS

Scenario

A new version of your application needs to be deployed without impacting users.

How would you achieve this?

Solution

Use ECS Rolling Update (or Blue/Green with CodeDeploy for advanced deployments).

Rolling Deployment

Version 1

↓

Deploy Version 2

↓

New Tasks Started

↓

Health Check Passed

↓

Old Tasks Terminated

Deployment Configuration Example:

Minimum Healthy Percent = 100

Maximum Percent = 200

This ensures:

- Existing tasks remain available
- New tasks start first
- No downtime

Rollback

If health checks fail:

- Deployment stops
- ECS retains healthy tasks
- Roll back to the previous task definition revision

Scenario 5: Securely Pass Database Credentials to ECS Containers

Scenario

The application requires database credentials. Storing them inside the Docker image is not allowed.

How would you securely provide credentials?

Solution

Use **AWS Secrets Manager** (or Systems Manager Parameter Store).

Flow

Secrets Manager

↓

IAM Task Role

↓

ECS Task

↓

Application

Steps

1. Store credentials in Secrets Manager.
 2. Grant the ECS Task Role permission to read the secret.
 3. Reference the secret in the Task Definition.
 4. ECS injects the value as an environment variable.
-

Benefits

- No hardcoded passwords
 - Automatic secret rotation support
 - IAM-based access control
 - Audit logging with CloudTrail
-

Scenario 6: Migrate ECS from EC2 Launch Type to Fargate

Scenario

Your company wants to eliminate EC2 instance management while continuing to run the same containerized applications.

How would you migrate?

Solution

Current

ECS Cluster

↓

EC2 Instances

↓

Containers

Target

ECS Cluster

↓

AWS Fargate

↓

Containers

Migration Steps

1. Ensure the application is stateless.
 2. Update the Task Definition:
 - Launch Type: Fargate
 - Network Mode: awsvpc
 - Specify supported CPU and Memory values.
 3. Create a new ECS Service using Fargate.
 4. Test connectivity through the ALB.
 5. Shift production traffic.
 6. Remove EC2 capacity after validation.
-

Benefits

- No server management

- Improved security isolation
 - Pay only for resources used
 - Simplified operations
-

Scenario 7: ECS Service Cannot Pull Docker Image from ECR

Scenario

Tasks fail immediately with an image pull error.

How would you troubleshoot?

Troubleshooting Steps

Verify Image

```
aws ecr describe-images
```

Confirm:

- Repository exists
 - Image tag is correct
-

Verify IAM Execution Role

The ECS task execution role requires permissions such as:

- `ecr:GetAuthorizationToken`
 - `ecr:BatchGetImage`
 - `ecr:GetDownloadUrlForLayer`
-

Check Networking

If tasks are in private subnets:

- Internet access via NAT Gateway, **or**
- VPC Interface Endpoints for Amazon ECR (API and DKR) and S3

Verify Task Definition

Check:

- Repository URI
- Image tag
- Region
- Account ID

Common Causes

- Wrong image tag
- Missing execution role permissions
- No internet/NAT or missing VPC endpoints
- Incorrect repository URI
- Image deleted

Scenario 8: One ECS Task Is Unhealthy While Others Are Healthy

Scenario

Only one task repeatedly fails health checks and gets replaced.

How would you investigate?

Investigation

Check ECS Events

Service Events

Look for:

Task failed ELB health check

Verify

- Health check endpoint
 - Container port
 - Application startup time
 - CPU usage
 - Memory usage
 - Application logs
 - Dependency failures (database, cache, external APIs)
-

CloudWatch Logs

Look for:

- Java exceptions
 - Node.js errors
 - Python tracebacks
 - OutOfMemory errors
 - Connection timeouts
-

Resolution

- Increase health check grace period if startup is slow.
 - Fix application issues.
 - Increase CPU or memory if resources are insufficient.
 - Validate container dependencies.
-

Scenario 9: Deploy Multiple Microservices on the Same ECS Cluster

Scenario

An e-commerce application consists of:

- User Service
- Product Service
- Order Service
- Payment Service

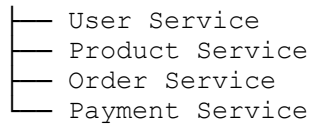
How would you deploy them?

Solution

Use a single ECS Cluster with separate services.

Architecture

ECS Cluster



Each service has:

- Its own Task Definition
- Independent Service
- Auto Scaling policy
- CloudWatch Logs
- IAM Task Role

ALB routes traffic using path-based routing:

/users

↓

User Service

/products

↓

Product Service

/orders

↓

Order Service

Benefits

- Independent deployments
 - Individual scaling
 - Better fault isolation
 - Easier maintenance
-

Scenario 10: ECS Deployment Failed in Production

Scenario

A new application version was deployed. After deployment:

- Health checks failed
- Users received HTTP 503 errors
- Deployment did not complete

What would you do?

Incident Response

Step 1: Check ECS Service Events

Look for:

`Task failed health check`

Step 2: Check CloudWatch Logs

Possible issues:

- Startup exception
 - Configuration error
 - Database unavailable
 - Missing environment variables
-

Step 3: Verify ALB

Check:

- Target Group health
 - Health check path
 - Port configuration
 - Success codes
-

Step 4: Roll Back

Update the ECS service to use the previous Task Definition revision:

Revision 18

↓

Rollback to

↓

Revision 17

The service launches healthy tasks from the earlier revision.

Step 5: Root Cause Analysis

Review:

- Recent code changes
- Docker image changes
- Configuration updates
- Secret changes
- Infrastructure modifications

Document:

- Root cause
 - Impact
 - Timeline
 - Corrective actions
 - Preventive actions
-

ECS Interview Cheat Sheet

Component	Purpose
ECS Cluster	Logical grouping of container infrastructure
Task Definition	Blueprint that defines container configuration
Task	A running instance of a task definition
Service	Maintains the desired number of running tasks
Launch Type	EC2 or Fargate
Task Execution Role	Allows ECS to pull images, write logs, and retrieve secrets
Task Role	IAM role assumed by the application inside the container
Application Load Balancer	Distributes traffic and performs health checks

Component	Purpose
Amazon ECR	Stores Docker container images
CloudWatch Logs	Centralized application and container logs
Service Auto Scaling	Automatically adjusts task count based on metrics
Service Discovery	Enables microservices to discover each other using DNS
Capacity Provider	Manages ECS capacity using EC2 Auto Scaling groups or Fargate/Fargate Spot